Design Considerations for J2EE Applications

The Java 2 Platform, Enterprise Edition (J2EE) delivers a whole range of useful technologies. So far in this book we have spent a great deal of time covering what these technologies are and how they should be applied.

> **However, the key to creating usable, flexible, and maintainable applications is to apply technologies that are appropriate to the *context* of the problem being solved.**

In this chapter we will look at some common ways of applying J2EE technologies to solve design issues in particular contexts. We will look at:

- What we mean by design and architecture
- The relationship between design and the context in which it occurs
- The forces at work when designing a typical e-commerce application for the J2EE platform
- Strategies for dealing with issues such as distribution, state management and transactions in J2EE application design
- The advantages and disadvantages of various J2EE technologies when applied in certain design contexts

The World is Changing

When software vendors who are usually at war agree on something, is this a good omen, or a bad one? Consider the 'enterprise' application platforms promoted by Sun and Microsoft. From a high level, the Java 2 Platform, Enterprise Edition (J2EE) and the Windows Distributed interNet Architecture (DNA) look very similar. Both combine multiple tiers, thin or thick clients, distributed object protocols, standard data access APIs, messaging services, middle-tier component environments, and transactions.

Is it a part of a secret joint marketing strategy, or is it simply that the world has changed and the changes are being reflected in the products on offer? The arrival of e-commerce and Internet timescales has changed the way that most business applications are defined and developed. The requirements are higher than before, and the timescales shorter. There is an increasing need for adaptability, since tomorrow's business requirements will almost certainly not be the same as today's. To design and develop applications under these conditions, you need serious (some would say 'professional') help.

From my perspective, this help takes two forms:

- A **standardized framework** on which applications can be built and deployed. The framework should provide appropriate levels of functionality and should also help to automate the creation of standard 'plumbing'.
- A set of **best practices** for using that framework. Most software developers do not have an infinite amount of time to spend on contemplating the philosophy of design or learning the most efficient ways of using certain APIs. What they need are guidelines to help them write good applications using the framework.

> **In Java terms, the framework for development of distributed business applications is J2EE. The features and functionality of the platform allow for the creation of flexible, scalable, distributed, multi-tier, component-based applications.**

In fact, the buzzwords listed above are just a few of those regularly applied to J2EE in an average marketing document. Basically, J2EE provides the landscape for the development of modern, Java-based enterprise and e-commerce applications.

Even the vendors now understand that developers need help in exploiting the ocean of functionality that has been delivered. Microsoft has provided, amongst other things, the Duwamish Books sample application to illustrate DNA best practices (`http://msdn.microsoft.com/voices/`). Sun Microsystems has recently issued the Sun Blueprints Design Guidelines for J2EE (formerly the J2EE Application Programming Model) and the associated Java Pet Store application to provide the same type of assistance (these can be found at `http://java.sun.com/j2ee/blueprints/`).

Obtaining the knowledge for effective application design on an enterprise platform does not come cheaply. The Sun Blueprints Design Guidelines for J2EE (hereafter referred to as the 'J2EE Blueprints') version 1.0 runs to around 350 pages. In addition, there is also a multitude of other resources, such as books, e-mail and news groups, newsletters, and articles that provide insights, discussion, advice, and sometimes controversial opinions on the creation of enterprise applications. Some of these are listed below:

- *Software Architecture in Practice*, Bass, Clements, and Kazman (ISBN 0-201-19930-0)

- *Client/Server Programming with Java and CORBA*, Orfali and Harkey, Published by John Wiley and Sons (ISBN 0-471-24578-X)
- *Programming with Enterprise JavaBeans, JTS and OTS*, Vogel and Rangarao, Published by John Wiley and Sons (ISBN 0-471-31972-4)
- *The Java 2 Enterprise Edition Developer's Guide* (referenced from the J2EE documentation page at `http://java.sun.com/j2ee/docs.html`)
- J2EE Patterns discussion e-mail list at Sun (`J2EEPATTERNS-INTEREST@JAVA.SUN.COM`)
- Interesting opinion in the ObjectWatch newsletter (`http://www.objectwatch.com`)

The combined thoughts and opinions found in these sources (and many more besides) cannot all be condensed into one chapter, hence the remainder of this chapter examines some of the main issues in enterprise development and the type of solutions that apply in a J2EE environment.

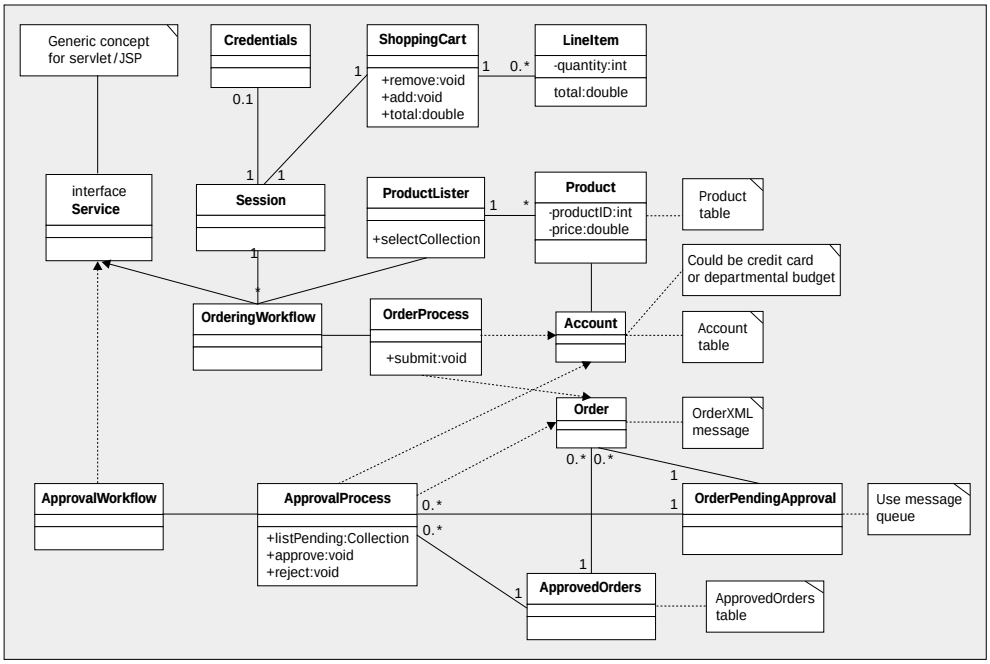
Design Context

Software is developed to solve a problem. That problem may be of a business nature, such as how to reduce the amount of time it takes an organization to process a customer's order, or it may be more technical, such as ensuring that a satellite control system responds correctly when instructed to change course. In either case, you cannot make an absolute judgment of the quality of the proposed solution. You can only judge the solution in terms of the context provided by the problem to be solved.

Consider the design for an airplane. Conventional wisdom would say that stability is a very important factor for an airplane design. However, many modern fighter aircraft are intentionally designed to be unstable, since this improves their maneuverability. This is a good design decision for a fighter plane, since a slight improvement in maneuverability can mean the difference between success and failure. Also, the organizations that use such planes are willing to pay for the expensive computer systems that help turn the unstable machine into something that can be controlled by a human pilot.

If you change the context of the problem, for example by looking at a passenger aircraft, safety considerations are paramount. Suddenly stability becomes very important – much more so than maneuverability. For such a plane, the extra maneuverability is not worth the risk to safety or the cost of the extra computer systems to control an inherently unstable plane. Hence, the same design decision in a slightly different context can be the wrong thing to do. No design decision is inherently good or inherently bad: it all depends on the context and the forces acting on the design. This applies both to the modeling of the proposed solution and the mapping of the model onto the underlying platform.

To discuss specific aspects of design then, it is useful to have a context in which to judge decisions. In this chapter, this context is provided by a simple e-commerce purchasing application. A first-pass UML model for the system is shown overleaf:



This is by no means the finished article, but provides a basis around which design decisions can be discussed. Indeed, some of the design decisions will affect how the model evolves. The initial stages of analysis and design involve the creation of a model that maps the problem to be solved. The inputs to the creation of this model are the use cases or user stories that describe the desired system functionality. These inputs can be crystallized in UML notation such as class diagrams and sequence diagrams. This initial problem domain model, which reflects 'real' things (people, documents, etc.), is then used as the basis on which to create a model of the proposed solution (the classes, components and interactions that will make up the software system). This solution will be shaped to a degree by the environment into which it will be deployed. In our case, this environment is the J2EE platform.

Consider a 'real' architect designing a house. The architect is given a set of requirements stating that the house must contain four bedrooms, a bathroom, living room, garage, and so on. If the house is to be built on the side of a hill, the design will differ from a house designed for a flat piece of ground. The design may make use of the slope of the ground to site the garage underneath the rest of the house, or it may build parts of the house into the hillside to improve the thermodynamic efficiency of the house.

Similarly in software, the solution that evolves from a basic model will differ depending on the *terrain* onto which it will be deployed. A pattern that is prevalent in this environment may suggest a convenient solution to a particular problem (such as the ground-floor garage described above). In this way, the environment will shape the model in the same way that the requirements do.

The evolved model can give us useful information about the system, such as the relationships between the different classes. This information is very useful when deciding some deployment issues such as componentization and packaging. Again, if there is a known constraint relating to the deployment of different parts of the system (for example, if one component must live on a particular database server) then this may mean that two related components cannot be packaged together. This in turn affects the relationship between the classes involved and thus affects the shape of the model. In this case, the interface of one or more of the classes involved may need to be altered to allow for the effects of distribution, or a proxy class may be introduced.

Before discussing the design decisions relating to our sample application for this chapter, let's first outline how the application is intended to work. The application will be referred to as the POSystem. Employees on a corporate intranet will use it to fill out purchase orders online. The user will select from the available products and build up an order, much like the usual web shopping cart. Once the user has selected the products they require, they will submit the order. At this point, the application diverges from web-based shopping since the user does not actually purchase the contents of their cart using a credit card. Instead, the purchase order that they have built up is sent to the appropriate departmental manager for approval. This approval will not happen in real time, since the manager may not be available. The second part to the application involves the manager processing the purchase orders that have been submitted and either approving or rejecting them. Another aspect of the context is that this system is intended for Acme Multinational Inc., and hence it has potentially thousands of concurrent users.

The design shown above for the system is reasonably simplistic and does not take into account the underlying technology landscape on which it will be implemented (although the comments do contain some thoughts and opinions). There are doubtless many aspects of the system that you might elaborate on, or where you may design things somewhat differently. Remember, however, that all solutions reflect the context of their design, and one of the main forces in this case is that the model is simple enough to be readily understood. The intention is to capture the *essence* of the system and not necessarily the detail. This is sufficient to discuss the move from model to system architecture. We do not need to build a completely working model, and hence become bogged down in the domain detail of purchasing systems. You can see a more involved, or evolved, solution for this type of problem in the Java Pet Store provided as part of the J2EE Blueprints.

A quick look at the model shows that the part of the system that builds the purchase order revolves around the `Product` class. `Product`s can be listed on some basis (product code, category) and presented to the user. The `Product`s selected by the user will be stored in a `ShoppingCart`, represented by a series of `LineItem`s. The interaction with the user will be governed by the `OrderingWorkflow`, which encapsulates the user interface logic for displaying products, selecting products,and submitting a purchase order. As part of the ordering process, the purchase order must be sent to the appropriate manager. This will require some form of identification of the user and also of the manager. Credentials can be gathered from the user for this purpose. When the purchase order is submitted, it could be checked against departmental limits of various sorts, including how much money is left in the departmental budget.

The purchase order produced will be stored somewhere, pending retrieval by the manager. The manager will progress through the `ApprovalWorkflow` user interface logic to view and approve or reject purchase orders that have been submitted.

This, in essence, is the application to be implemented. There are many decisions that must be taken to evolve this model into an implementation using J2EE technologies.

On Architecture and Design

Some astute readers will have noticed that I have rarely used the word *architecture* so far. This is entirely intentional, since the use of the terms 'architect' and 'architecture' in the world of software is the subject of much debate. In building terms, the role of an architect is reasonably well understood. In software terms, application architecture can mean different things to different people. This then leads to the question of where architecture stops and where design begins. Are they, in fact, separate concepts, or just two facets of the same one?

The word 'architect' is derived from the ancient Greek for 'master builder'. If we take this at face value, there is the implication that an architect has mastered the craft of building to the degree that they can identify best practices and common patterns. It also implies that they understand many of the constraints imposed by the real world of bricks, stone, and concrete. This would be a good analogy to use in our current context, with the J2EE platform APIs providing the bricks.

I do not want to get bogged down in 'religious' discussion about software architecture. However, discussion of the term tends to revolve around certain aspects of a system and these are important for us to consider, namely:

- The system components that perform the business tasks of the system
- The type of interaction between those system components
- The services used by those system components
- The underlying platform that delivers or supports those services
- The style or idiom reflected by the system or its parts
- Other characteristics or capabilities (such as scalability) which address the non-functional requirements of the system

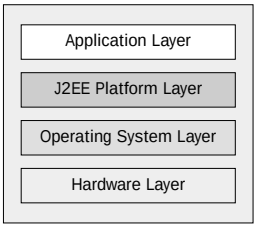
When considering these topics I will use architecture in its general meaning of structure or form.

Architectural Styles

People will refer to 'Service-Based' architectures and 'Layered' or 'Tiered' architectures. However, it is not a case of making a straight choice between them. Many systems use aspects of multiple architectural styles in order to solve different parts of their overall problem.

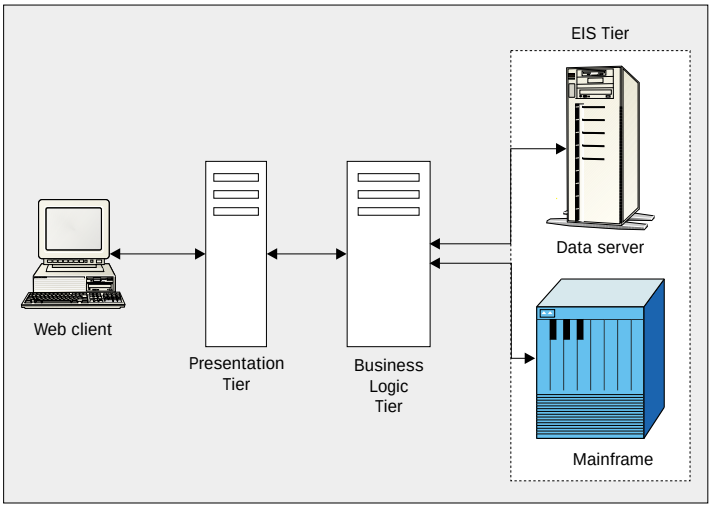
Layered and tiered architectures have much in common. In many senses, tiered architectures can be viewed as a particular form of layered architecture. The main aim is to abstract some elements of a system in order to simplify the overall structure. A layer represents a cluster of functionality, or components that have similar characteristics (for example, type of function or physical location). Each layer provides functionality to the layers around it.

When discussing layered architectures, many people will think of top-to-bottom layers, such as those seen in a network stack. In the ISO model of a network stack, for example, the transport layer makes use of functionality provided by the network layer, which in turn uses the functionality of the data-link layer. Such layering provides abstraction of the underlying layers to allow for substitution. In terms of our application, the J2EE platform provides multiple layers – hardware, operating system, and J2EE itself – as shown in this diagram:



Since the application sits on top of the J2EE platform, the underlying hardware or operating system can be changed to provide better characteristics (faster, cheaper, more stable, etc.) without the need to re-write parts of the application.

Tiered architectures are a specific type of layered architecture based on a user-focused view of the system. This leads to a front-to-back partitioning with the user at the front and the underlying data etc. at the back. In a typical 3-tiered architecture, the user interface components will be in a separate tier from the business logic components, which in turn are in a separate tier from the data access components. The tiers reflect some form of physical partitioning, so that the three sets of components will exist in different processes, and typically on different machines. Communication will flow between the various tiers as the application goes about its work. A web-based 3-tier architecture can be seen below:



The POSystem application would use a tiered architecture for flexibility and scalability reasons. The separation of the user interface, business logic, and data access allows us to substitute better products or techniques at each tier without disturbing the others. The decoupling between the tiers allows us to add more capability to each tier as the system scales. An example would be adding more web servers at the user interface tier to service more clients. This would not automatically require more database servers on the data tier (although it might!).

A service-based architecture views the components of a system in terms of black-box services. An example from the system level would be a transaction service, such as the **Java Transaction API (JTA)** provided by J2EE. In this case, the user of the service is solely concerned with the interface to and characteristics of the service, not its implementation. Service-based architectures can be viewed as a larger form of component architectures, since applications will be created by writing programs or scripts that call upon various services to perform their required tasks. The concept of a service starts to get away from the concept of fixed clients and servers, since services will call other services to perform their tasks. There is no implicit requirement for calls to flow in a particular direction.

In terms of the POSystem, we could define a catalog service, a PO submission service, an account service, an ordering service, and so on. The user would use the ordering service, which would in turn use the catalog, account, and PO submission services to perform its workflow.

Forces, Patterns, and Iteration

From the discussion so far, it should be clear that the architecture of an application needs to be designed. Design can be considered as the solution to a given problem, in a given context. The context will consist of a series of forces that must be understood and balanced by the designer. There will be various forces at work constraining enterprise architecture, such as:

- The business problem being modeled
- Required technologies, including legacy systems
- System capabilities (scalability, availability, etc.)
- Reach of the application (usually embodied as thick vs. thin clients)

All of these forces, and many more, will impact on the eventual shape of the system and the architecture that best suits it. As discussed earlier, good architecture is specific to the situation. Its usefulness depends entirely on its context; that is, on the problems addressed and the forces at work in the system. When defining the requirements for the POSystem earlier, various forces were specified such as the potential for thousands of users, the need to process purchase orders asynchronously, and the desire to implement the system on top of J2EE.

The forces on the evolution of the system can be functional or non-functional, and can vary from high- to low-level. As this implies, design must occur at many levels and stages:

- The solution model (although it may be called 'elaboration' of the problem model)
- The broad system architecture
- Interfaces of system components
- Interactions between system components
- Internals of system components

Does this imply that architecture is an output of design? Hopefully, the answer to this is 'yes'. However, architecture can evolve rather than be designed, but designing an architecture implies intent.

Note also that **patterns** of varying forms appear at all these different levels. A pattern is a proven solution to a problem in a given context. In software terms, patterns are essentially the distillation of the 'wisdom' gained by practitioners of what works well when specifying, designing, and implementing software. The patterns movement was initially popularized by the book *Design Patterns – Elements of Reusable Object-Oriented Software*, Gamma *et. al.*, Addison-Wesley, ISBN 0-201-63361-2. However, patterns are not solely applied in the realms of micro-architecture as described in this book.

Patterns can be found in many areas of software and systems. At a high level, patterns can be found when performing analysis in specific domains. The entities and relationships discovered during such analysis will be repeated across a business sector, and this repetition leads to the discovery of such patterns. This type of pattern is discussed in detail in Martin Fowler's book on analysis patterns (*Analysis Patterns – Reusable Object Models*, Fowler, Addison-Wesley, ISBN 0-201-89542-0). If you are working in a specific domain, such as finance or telecommunications, it would be worthwhile investigating the existence of domain-specific patterns to save time and effort. In UML, as in XML, the world does not need another purchase order.

At the architectural level, work is currently being done investigating patterns that apply specifically to J2EE. Sun has an e-mail discussion forum for such patterns (`J2EEPATTERNS-INTEREST@JAVA.SUN.COM`) and has delivered some of their own material on this subject (see the slides for session TS-1341 *Prototyping Patterns for the J2EE(TM) Platform* at JavaOne 2000, `http://java.sun.com/javaone`). Other, Java-specific patterns exist at a variety of levels all the way down to language idioms. There are various Java patterns books on the market and some good discussions and articles, such as JavaReport's *Patterns in Java* column (archive versions of these are due to be available at `http://www.curbralan.com`). Although not specifically Java-oriented, the book *Pattern Oriented Software Architecture: A System of Patterns,* Buschmann *et. al.*, Published by John Wiley and Sons ISBN 0-471-95869-7 gives a good view of patterns at different levels of a system.

As we discuss the options for the design of the POSystem, various patterns will present themselves as suitable solutions for some of the problems we will encounter. An example of this is the use of the Model-View-Controller (MVC) pattern for the separation of user interface from data, we will see later in the chapter.

The effects of high-level architectural decisions cascade down through the design. If the need for scalability is translated into the use of a stateless model (as in the example we will see later), then this in itself may affect lower-level design decisions (such as the use of stateless session EJBs). Semi-functional requirements such as extensibility may have a wide-ranging impact on design decisions throughout the system. Taking the example of extensibility, if a system currently uses two possible data exchange mechanisms but it is anticipated that more will be required, an extra layer could be created to accommodate the required extensions.

Similarly, other decisions will be made further down in the application hierarchy for reasons of efficiency or suitability to the underlying platform. These decisions can require changes to higher-level parts of the architecture and hence feedback loops will form. Most mainstream formal development processes, such as the Rational Unified Process (`http://www.rational.com/products/rup/`), now encompass an idea of iteration and feedback between the phases of a software development. Implementation issues or changing technologies may generate valid feedback into the higher-level design or analysis.

Remember that most problems have multiple solutions. If the feedback changes the forces in operation at a certain point, a different decision may well be more appropriate. For example, it may be found that the need for a `LineItem` to refer to a `Product` in order to retrieve the item price leads to inefficiencies that slow the system down. It may be decided to replicate this price information in the `LineItem` itself in order to reduce these inefficiencies. This will change the model of the POSystem shown previously, but the overall functionality is unchanged.

Many practicing software developers will not have time to keep track of informed debate about the nature of design and architecture. Even if they do, the hope is to extract something of concrete use which will help the developer make better or more informed choices next time they sit down to design a system.

At what level does this chapter cover design issues? Essentially, it is a reflection of some considerations, trade offs, best practices, and rules of thumb commonly encountered when mapping some form of higher-level model onto the J2EE platform. This, then, is the design that sits between analysis and implementation.

Some of this design can be independent of implementation technology. As a UML model is refined, some of those refinements work well across all potential architectures. Other refinements must be informed by the platform onto which they will be deployed. The design of the system must take into consideration the idioms of the platform. It must also try not to fight the platform, going "against the grain". This, for me, is the main issue in this type of discussion.

The key challenge for many projects is performing this evolution from the 'pictures' to the code.

We can now look at some general design considerations for J2EE-style environments and at some J2EE-specific idioms for design. One of the non-functional requirements stated above for our POSystem is that the implementation must be Java-based and must use the J2EE platform. This does at least provide us with some stakes in the ground.

Distributed Design

Distribution lies at the heart of J2EE applications, and distributing functionality provides much of the flexibility in the J2EE platform. However, analysis and high-level design frequently does not consider distribution, seeking first to create the correct business components. In this ideal world, all access between clients and servers would be transparent, as if the services were located on the same machine. Distribution also introduces complexity in the form of additional method calls and different programming paradigms. This much should become clear from reading Chapters 2, 4 and 22. In a distributed environment method calls are no longer deterministic, as they are in local processing. For example:

- A call may fail due to a network-related error
- Partial failure of a networked operation can cause problems, and this must be detected and corrected by the application
- Timing and sequencing issues arise and there is no guarantee of the order in which a sequence of calls from multiple processes will be received

Other factors must also be considered:

- Many orders of magnitude increase in the overhead of method calls
- The potential for communication to dominate computation
- Implicit concurrency in the system
- The need to repeatedly locate remote components as they migrate between servers over time
- The levels of unpredictability make it difficult or impossible to guarantee the consistency of all the data in the system at any one time

These and other issues must be considered when designing solutions for distributed environments. A good, if slightly dated, discussion of the issues with distribution can be found in the paper *A Note on Distributed Computing*, Jim Waldo *et. al.*, 1994, `http://www.sun.com/research/techrep/1994/abstract-29.html`.

In the context of our POSystem application, we must ensure that we take the following steps:

- Partition the system well so that components that communicate frequently and in bulk are co-located on the same machine where possible. Since communication over a network is comparatively slow, complex, and expensive we should seek to remove it where possible.

In the POSystem, it would not make sense for the `ShoppingCart` to be on a different machine from the `OrderingWorkflow`. This would unnecessarily increase the complexity of the interaction between these components.

- Where communication over a network is unavoidable, the interactions and method calls must be designed in a network-friendly way.

  If our system is large, it is almost certain that the `OrderingWorkflow` will be on a different machine to the `ProductLister`. Some of the steps that can be taken to improve the communication between these components, such as batched methods or the use of serialized JavaBeans to hold data snapshots, are discussed later in this chapter.

Developing distributed systems is hard. Don't believe any product vendor who tells you otherwise. It is important to have a good understanding of what is going on in the system and what sorts of tradeoffs are being made. However, this does not necessarily mean that you need to write every line of code yourself. The type of distributed middleware typified by J2EE helps the designer and developer by providing much of the infrastructure and 'glue code' required when developing distributed systems. The benefits of these contracts, services and the use of interposition are discussed in Chapter 18.

The need to deliver performance, scalability, and data integrity in distributed systems underlies much of the functionality delivered in J2EE. Some examples are:

- Containers that can control concurrency and optimize performance while maintaining isolation
- Distributed transactions for data integrity
- Message passing to improve performance, scalability, and fault tolerance
- A naming service that provides location-independence

Start at the Beginning

Since we must start somewhere, let us start with the first thing the user needs: the listed products. All of the product listing functionality could be encapsulated within one or more JavaServer Pages or servlets. I will try to avoid discussion of any of the basics of JSPs and servlets since these have been well covered in Chapters 7 to 15. However, let's look at some aspects again from a design viewpoint.

Using JSPs or servlets would satisfy the requirements for this part of the system, since the product data could be accessed directly in the database via JDBC. JSPs and servlets give us the ability to implement the user workflow associated with selecting categories and products to list. Even at this simple stage, design choices can be made to create a more flexible and maintainable system. One aspect is the use of database connection pools to aid scalability. The whole issue of scalability and resource recycling is discussed in later in this chapter.

As we discussed in Chapter 14, the main design imperative for JSPs is to remove as much of the code as possible from the page. This separation of the presentation and the code has many benefits, such as:

- Code can be used in multiple pages

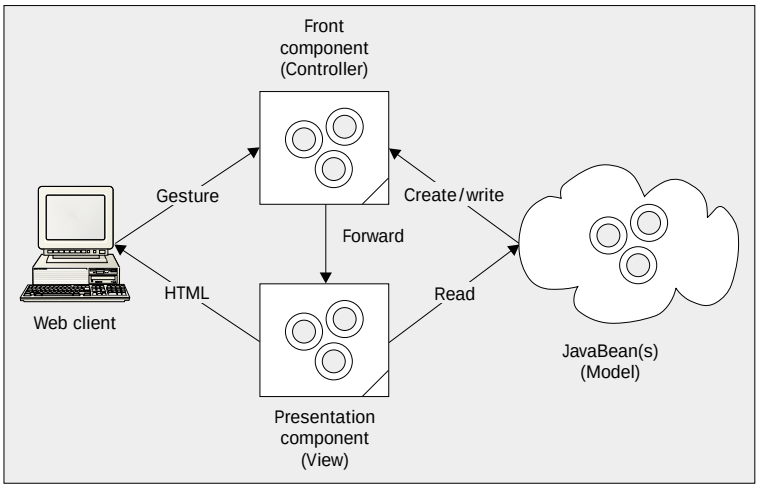
- Good design should mean that the page designer does not have to work around chunks of Java code
- Changes to the workings of the code do not require the pages that use that code to be edited

This guarantees that any use of JSPs beyond the most simplistic will involve such partitioning of HTML and Java code. As we saw, this partitioning can take various forms:

- The code for the page can be implemented as JavaBean components. Some logic remains in the page to drive the JavaBeans at the appropriate points.
- Code can be further abstracted into tag libraries. (For more information, see Chapters 12 and 13.) Use of custom tags makes it easier for a UI designer to manipulate the page.
- Functionality can be concentrated into one or more servlets (or JSPs) that are not involved in display. These will forward the results of their processing to JSPs that are dedicated to display.

While all of these mechanisms will reduce the amount of code required in our JSPs, from a design perspective, the last one, involving the use of **front components**, is the most interesting. A front component is a JSP or servlet whose sole task in life is to process user input (sometimes called user gestures). It contains no code to generate output for the user, however it captures data from the user, processes it, and decides what should be done next. The processed data is then passed to an appropriate JSP that is entirely concerned with formatting and presenting the data it is given.

Use of front components is part of the way towards the classic Model-View-Controller (MVC) architecture, with the front component acting as the controller, the presentation component as the view, and the data or state part of the system as the model:



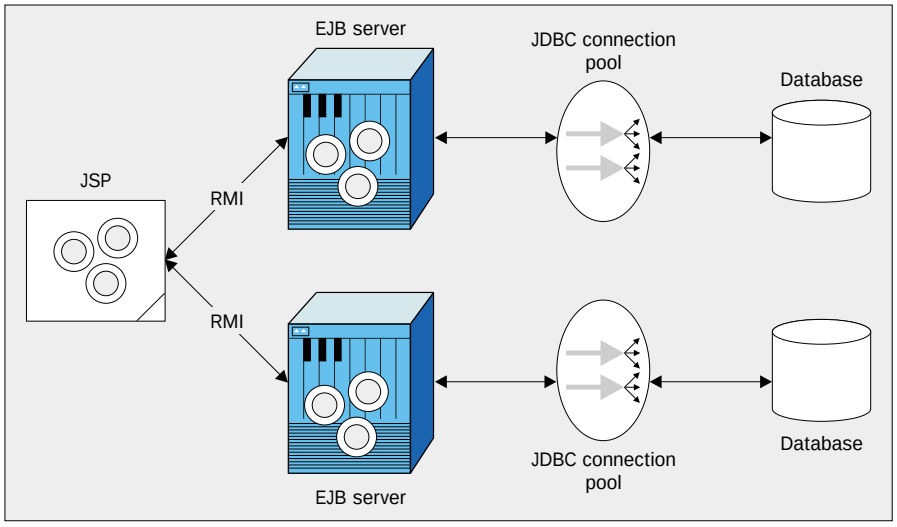
Using this architecture gives us some worthwhile advantages. Changes to the user workflow would not necessarily require changes to the view or model. Similarly, changes to the data access mechanism can be transparent to the view and controller. The three way separation gives a large number of options should changes be required in the style or location of the user interface. If the client had to be re-implemented as a Swing UI, the model could remain the same and simply be relocated to the client. If the workflow logic of the controller were implemented as a separate class from the servlet, then this too would be relocatable quite easily. Alternatively, a proxy could be created for use on the client that would map the Swing input to the appropriate user gestures expected by the JSP/servlet controller. The view could then be substituted for one that generated XML instead of HTML. The proxy on the client could then use this output to update its local model.

There is, of course, one slight problem with one of the suggested strategies above. Moving the model to the client is not quite the simple task that some 'draggy-droppy-pointy-clicky' tools would have you believe. If our model is accessing the database via JDBC, moving this functionality from server to client has some serious accessibility and security issues. Indeed, even moving it from server-to-server may cause such problems. This is just one of the issues with a JavaBeans-based model. If we were creating an application that had to be highly scalable, then this architecture would generally prove to be sub-optimal.

Holding state on the server on behalf of a client is a tricky business. In this case, the model will contain data and, potentially, a JDBC connection to the underlying database through which it can refresh its contents. By now, you should be well aware that this is simply not scalable. Consider the following issues associated with this scenario:

- To solve the memory problems, it may be possible to keep adding more memory. As the physical memory limit of the hardware is reached, another server can be added to handle some of the clients. The immediate problem here is that we then get lock-in between clients and servers. A particular server will hold the model for a particular client. This can potentially work against any load-balancing strategies used by the system.
- Physical memory is only one of the resources used by an application. In the case of the model, it also requires a database connection to access its underlying data. While memory may be relatively cheap, database licenses tend not to be. If each model holds one of these resources, they will be exhausted fairly quickly. If it must create one every time it needs to refresh or update its data, performance will be impaired.

If you have read the earlier chapters on EJB (Chapters 18 to 23), you will probably know where this is leading. To achieve serious scalability for most J2EE applications you should introduce EJB into the architecture, as shown below:



The J2EE Blueprints acknowledge that some applications will begin from a simple base HTML/JSP and then evolve as the requirements on the system change. In fact, they provide what might be thought of as a 'maturity model' for web-based J2EE applications, as shown here:

Application Type	Technologies	Complexity	Functionality	Robustness
HTML Pages	HTML pages	Very low	Very low	High
Basic JSP Pages and Servlets	HTML pages JSP pages Servlets	Low	Low	Low
JSP Pages with Modular Components	HTML pages JSP pages Servlets JavaBeans components Custom tags	Medium	Medium	Medium
JSP Pages with Modular Components and Enterprise JavaBeans	HTML pages JSP pages Servlets JavaBeans components Custom tags Templates Enterprise JavaBeans	High	High	High

This model acts as a good rule of thumb when considering which technologies will be required to implement a particular application. However, be aware that there will be exceptions. Some very scalable applications can be built simply with servlets, as long as you use the right server products.

Adding the Middle Tier

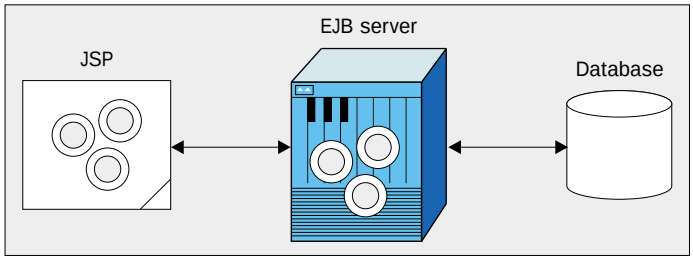
Our MVC system has expanded somewhat, since the state it uses now resides on the middle tier in an EJB. This presents us with several new design issues that must be addressed:

- What type of EJB should we use to provide the data access? Should we model the database as an entity EJB, or should we use a session EJB and fire SQL queries at the database from there? The pros and cons of the different types of EJB are discussed in Chapters 18 to 20. What we need to do is consider some of the patterns discussed there, in the context of our application.
- How will the web-tier components interact with the EJBs? Although we are gaining scalability and flexibility when using EJBs, we are potentially incurring a considerable communication overhead.

Think back to the application being used as the context for this design consideration. As a first pass, we might want to model our `Product` data as entity EJBs. This would mean that we could take advantage of Container-Managed Persistence (CMP) to save us writing some JDBC code and allow the container to optimize data access. Our web-tier components would then use the entity's home interface to search for appropriate `Product`s to display, based on the user's preferences.

The immediate danger here is that this will degenerate into a frenzy of network-unfriendly property-based programming. Use of multiple getter or setter methods on distributed objects leads to an ugly 'sawtooth' effect in which communication dominates computation. It also increases the risk of partial failure should there be a problem part way through the getter/setter sequence. The first step to combat this is to use a `Serializable` JavaBean, say `ProductDetails`, to represent a snapshot of the data held in the entity. This use of the 'Combined Attributes' pattern (or distributed programming idiom) means that only a single method call is needed to retrieve all the data from the EJB.

`ProductDetails` would provide us a certain amount of optimization; however, the 'sawtooth' will quickly reappear if we have a `Collection` of `Product`s returned to the web-tier, from which data must be extracted. We quickly arrive at a commonly found EJB form of the Façade pattern. This has a session bean act as a façade or controller for a particular type of entity bean. Our web-tier components will converse with the session bean, `ProductLister`, which interacts with entities behind it. It will perform any entity searches and method calls on our behalf and will return to us just a collection of `ProductDetails` beans. This means that not only is no data access performed across the network, but the iteration is local to the web-tier also. All calls to the entities can be performed from within the same EJB server for optimization. The use of a session bean façade is shown below:



There are still certain considerations here. The `ProductDetails` bean need not contain the whole of the data in the `Product`. As a general rule, the data in such beans should be read-only. Since it represents a snapshot of the data, there may be limited value in including volatile data in the bean. If, for example, the `Product` contained the current stock level for that product, it may not be appropriate to include it in the `ProductDetails` bean since such information may be out of date by the time it is used. Such information should be retrieved 'fresh' directly from the `Product` (or its session façade) when needed. Another thing to bear in mind when using this data bean paradigm is that of interoperability. In the Java-to-Java scenario, it is easy to pass such serialized beans through an RMI interface. However, if any of your clients are potentially CORBA clients, then this interface could potentially be off-limits to them. Even if the client ORB implements the objects-by-value functionality required by CORBA 3, it still implies that more work needs to be done when unmarshaling the bean on the client. Integration issues with CORBA are discussed in detail in Chapter 29.

The use of a data bean to represent the EJB on the client points to a common pattern of design for J2EE. This involves the splitting of 'logical' data or functionality between tiers. For data, this means the use of a data bean on the web-tier as a representation of the data in the EJB-tier. This is a good strategy for providing quick access to slow-moving data. Similarly, the processing of user gestures may be split between tiers. A servlet on the web-tier may perform initial processing on the user input and then hand its data back to a session EJB, which can encapsulate many of the workflow rules that govern the flow of the user through the application. In this way, multiple types of client on other tiers can easily use that same session EJB.

The final question relating to the `Product` is whether it should, in fact, be modeled as an entity at all. Given that clients will use its session façade to access its data, would we not be better off accessing the database directly from the session bean and forgetting about using an entity bean? There is no absolute right or wrong answer here, although there should be a preference for abstraction over direct access. Much will depend on the origin of the data, the efficiency of the database, the efficiency of the EJB container, whether the entity's data is used by multiple beans on the EJB-tier, and the communication overhead between the EJB container and the database. A big deciding factor here can be the need for transactions. Entity beans will provide transaction control over the data in a database-independent way. However, if the data in our `Product` bean cannot be updated, then there is no need for transaction support. So, for example, the decision about whether to include the stock level in the `Product` class in our UML model would be very relevant here.

Going Shopping

Well, that concludes the consideration of the issues surrounding the listing of the products in our catalog. Now we need to actually do something useful, like ordering some! Creating a purchase order has a particular workflow associated with it:

- Show list of products to user
- Allow user to select one or more products from the list
- Submit the order

This will have iterations and other conditions, but it captures the essence of what we are trying to do. This workflow is encapsulated by `OrderingWorkflow` on the POSystem UML class diagram, and forms the controller for the form of MVC described above. The `Product` (and its associated session bean) takes on the role of the model. The type of view will depend on whether we are using a thick or thin client.

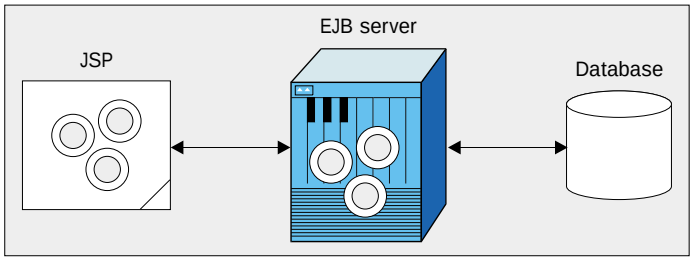
Our workflow is best implemented as a session EJB. This provides the best solution for scalability and flexibility. As mentioned previously in the discussion on splitting functionality, the implementation of the controller will usually be split so that the layer nearest the client will handle the user gestures, including any input error handling. This role will be performed in the web-tier for a thin client and in the client itself for a thick client. The user gestures will then be translated into an associated 'logical event' and passed to the workflow session bean that will then perform the associated business logic.

> *It is worth re-iterating here that this design is not prescriptive. The decisions I am making here are based on my own opinions of how best to solve the problem for my big Acme Multinational Inc. For Fred Bloggs Insurance Brokers Ltd. (your friendly local broker), encapsulating the workflow in a servlet may provide all the scalability and flexibility needed, while reducing the overall complexity of the system. One of the principles of* Extreme Programming Explained – Embrace Change*, Kent Beck, Addison-Wesley, ISBN 0-201-61641-6 is to design for today's needs and not prematurely anticipate requirements that may never come to pass. As another designer puts it:*
>
> *"A simple, inelegant design concept we've never been able to convey to some of our in-house customers is to let us develop an application that will satisfy 90% of their needs but takes only a tenth the time to develop, rather than one satisfying 100% of their needs but taking ten times the development effort. If they think one possible permutation of data won't be supported,* that's *the one element that Suddenly Becomes More Critical to Life than Oxygen."*

Examination of the UML class diagram will show that the `OrderingWorkflow` class uses a `Session` class to hold the client's state, such as the contents of the shopping cart. Anyone who has worked with JSPs or servlets will immediately recognize this concept from the in-built session object provided by the servlet API, as described in Chapter 9. However, caution should be exercised here. The class represents the *concept* of a session, and does not necessarily map onto the servlet session implementation. When previously discussing the holding of state on the server, several issues were highlighted about storing state in this client-specific way. The management of memory, recycling of resources, and avoidance of server lock-in are trickier in the web container than in the EJB container. By using EJBs for our session and shopping cart functionality, we can ensure that resources are used efficiently. It is also more flexible, in that it can be used by both thick and thin clients.

Given that we are now potentially working with several session beans, it is worth quickly digressing into considerations of stateful versus stateless models. As we saw in Chapter 19, under the stateful model a server-side component instance is dedicated to one particular client, and holds its data in temporary storage for ease of access. Under the stateless model, on the other hand, there is no dedicated server-side component instance for the client and all data must be retrieved from persistent storage. Although it is the subject of many debates and arguments, it is commonly accepted that using a stateless model has better scalability characteristics than a stateful model. However, the stateless model requires more work on the part of the designer and client application. This impacts on our choice of session bean for our `OrderingWorkflow` and `ProductLister` classes. As it stands, the work of the `ProductLister` is entirely stateless – it is presented with a category and it lists the product within it. In accordance with good practice, the results would be returned as a collection of data beans leaving no state on the server. Hence, this would suggest that the `ProductLister` would work well as a stateless session bean.

`OrderingWorkflow`, on the other hand, must have some concept of state to understand where the client currently is in the ordering process. This opens up a key design question over where the state should be stored – with the client, in the EJB, or in a database:



Each of these options has both benefits and issues:

- **State at the client**
  The client must pass the state to each method call that needs it. This allows for stateless EJBs and so improves scalability. However, it puts more of a load on the client and the network, and can cause a security risk if done through cookies for thin clients. It also complicates the design of the EJB's remote interface.

- **State in the EJB**
  By using a stateful session EJB, the required state can be loaded or acquired over time. This simplifies the design of the EJB's remote interface, and lightens the load on the client and network. However, it does reduce the ability of the EJB server to reuse beans and can also lead to the client being locked into one server.

If the system is to support thin clients, then some method of associating the client with the appropriate EJB must be provided. The client is then issued with a cookie that identifies the EJB containing its state. This mapping must be handled by the JSP providing the user interface.

- **State in a database**
  The state for a stateless EJB can be stored in a database. The client is then issued with a cookie or identifier that identifies the state (usually the primary key of the state in the database). This has less of an impact in interface design than holding the state at the client, but has a small impact on the network load and client complexity. It does however retain the scalability of a stateless implementation, at the cost of more time spent looking up and retrieving the data from the database. Remember, though, that using the database cache or modeling the state as an entity bean could reduce this significantly. The term 'database' here really refers to any server-side repository, ideally a transactional one. However, if the state is not intended to be persistent then alternative storage, such as a JNDI-compliant directory service, would work well.

Scalability and performance requirements will have a large role to play in this decision. Once the state location is determined, it will have an impact on the design of the remote interface for the session EJB.

Submitting and Processing the Order

Once the user has progressed through the workflow for product selection, they will be ready to submit their completed purchase order for approval. The `OrderProcess` class, in our UML diagram, would have various responsibilities for checking the order. In addition to checking the validity of the order itself, it may also check to see whether the department has sufficient budget left for such an order or it may check to see if that particular user is allowed to raise purchase orders of that value. Since it is applying business rules to state that is passed to it, the `OrderProcess` class would be best mapped to a stateless session bean. Once the `OrderProcess` is satisfied with the order, it must forward it for approval.

At this point, a major issue in distributed design appears. The order must be approved by the user's manager, but it is not possible to include the manager in the system. At this point the system becomes **asynchronous**. The purchasing process cannot block on a synchronous call waiting for the manager to approve the purchase order. Rather, the purchase order must be captured and stored to wait until the manager can examine and approve it.

All of the interactions we have dealt with so far (RMI and HTTP) have been synchronous in nature, and the architecture for this asynchronous requirement will have to use a different technology or approach. The mechanism used will depend on the tightness of the coupling between the ordering system and the approval system. Three possibilities suggest themselves:

- **e-mail**
  The application could use the JavaMail API to send an e-mail containing the purchase order to the manager. This would be very flexible for the manager, since they could even approve it from their web-based e-mail account while on vacation (or maybe not!). However, it does present more issues around the processing of the approval or rejection. Firstly, the application must be able to monitor the reply to the e-mail and process it when it arrives. The purchase order information must have been presented to the manager in human-readable form. To recover the original order information,

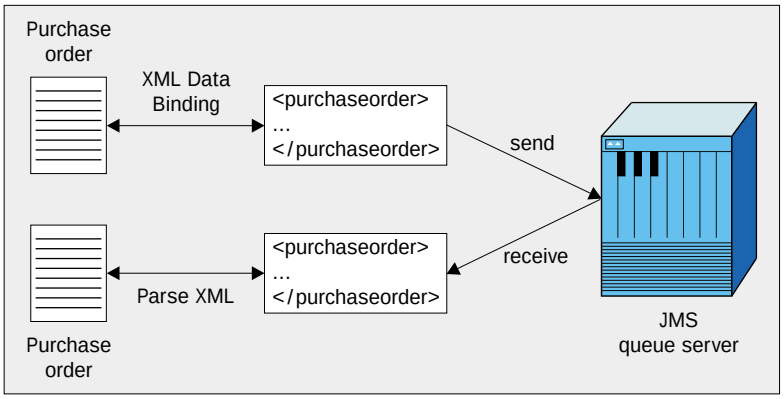
the application must either parse the e-mail message for all the order information or at least to recover some form of order identifier. An order identifier would refer to the original order data stored in a database somewhere. To provide authentication, the manager would have to use a digital certificate to sign the e-mail reply.

This solution gives good 'reach' and is very loosely coupled. However, it is somewhat unwieldy and potentially insecure.

- **Database**
  The order could be stored in a database within the system. The manager would then have to access some form of application that queried the database for orders awaiting that manager's approval. The manager would then interact with that application to approve or reject the orders. Since all of the information is stored in a database, there would be no issue with handling a return value as with e-mail. However, some way would be required of indicating that the order had been approved or rejected, such as a flag in the database or by moving it to another table. Security would be provided by authenticating the manager before they used the application.

  This solution has less 'reach', since it ties the two parts of the application to a common database (this may be required anyway to access product and budgetary information). This is the least asynchronous solution since the manager will have to go and access the data in the database, even if they are notified by e mail of its arrival. It is also tightly coupled to the order representation. However, it has good security and maintains a common development paradigm.

- **Messaging System**
  The Java Message Service (JMS) could be used to send an asynchronous message to the manager via a dedicated messaging system. This message would wait on an appropriate queue until it could be delivered to (or was retrieved by) the approval application. Since the message is not intended to be human readable, it could be encoded in a convenient form, such as XML or a serialized JavaBean. The serialized bean would be more convenient to use for completely Java solutions, but XML is more flexible and aids integration as described in Chapter 28. The arrival of XML Data Binding functionality should make XML an even better choice here, as shown below:



Once the order is approved or rejected, it could be moved to a specific 'approved' or 'rejected' queue. The design choice here is between the status of the purchase order being signified by its location (that is, which queue it is in) or by a flag in the message itself. Note that use of transacted queues would make sure that no message was left in limbo should the operation fail for whatever reason. Once in the appropriate queue, it would be available for further processing.

> This solution has a reasonable 'reach', since messages can potentially even cross boundaries between organizations. The flexible message format provides low coupling between the applications and security is included as part of the messaging system. Downsides include the need for the developer to learn a new API and paradigm for programming.

Again, the precise requirement of the application would govern the forces on this decision. Given the requirements of the POSystem, I would probably choose the JMS-based messaging solution with XML-encoded messages for a balance between ease of processing, integration characteristics, and potentially global reach for Acme Corp.'s remote sites.

As we have seen, the timeframe between the submission of the purchase order and its approval or rejection by the manager can be very long (a matter of weeks if they are on vacation). This is obviously unacceptable in terms of an e-commerce application. Indeed, for the typical transactional e-commerce system, a delay in the order of minutes or tens of seconds during the possessing of a user order can lead to backlogs and 'system busy' screens. For this reason, even some things that might intuitively be thought of as synchronous may be performed asynchronously. Take online credit card processing as an example. Since the credit card must be checked and approved by a credit card processor (typically a third party company), this approval call can be queued for later delivery. Calls can be made in a batch, or when the system has spare capacity. On the positive side this can improve the performance and scalability of the system immensely, since no waiting is involved. The only real downside is for any credit card holder whose verification fails. In this case, they must be informed later (typically via e-mail) that their transaction could not proceed.

As you can see, there are some advantages to using asynchronous operation and messaging systems. There are, however, several downsides. The first is that there is no return value from an asynchronous operation. If an output is needed, the sender and receiver must agree that the result will be posted on a particular queue. They must also agree to an identifier for the particular message to which it is a response. Finally, the sender of the original message must listen for message deliveries or poll the queue to await the arrival of their response. Another downside is that transactions effectively stop at asynchronous boundaries. While this makes a lot of sense (transactions must complete as quickly as possible), it does mean that other mechanisms are required to handle any failure of the task that the message represents. In the credit card case, the order must be cancelled and an e mail message sent to the user. Both of these actions take place outside the scope of the transaction within which the user ordered their goods.

Back at the POSystem, the `ApprovalWorkflow` and `ApprovalProcess` are analogous to the `OrderingWorkflow` and `OrderProcess`. They are subject to many of the same forces on their design and hence would be implemented in generally the same way. One major difference is that there is only one type of data being handled (`Order`s). All of the data being worked on is to be processed by this part of the application. There is no 'constant' data, such as the product catalog. Another difference is that the workflow associated with accepting or rejecting an order is very linear. There is no separate iteration phase, as there is when loading the shopping cart, followed by submission. Because of these factors, `ApprovalProcess` includes control of the data used during approval. To facilitate this, it has a method to list the purchase orders currently awaiting the manager's approval. This simplifies the overall shape of this part of the system.

The `Account` class used in the approval process would check any requests against departmental limits and rules to ensure that the manager conformed to, say, their maximum debit. The `Account` would be modeled as an entity bean. This is justifiable since `Account` contains business logic as well as data. The debit of funds from the department's account would be coupled with the approval of the order. In a more closely coupled system, the approval process may also decrement the stock level of each product held in a database. In both cases `Account` will form part of a transaction, since the debit must only happen if the order is successfully approved (that is, moved onto the 'approved' queue).

The need for a transaction by the approval process could usually be discovered from the associated use case when modeling the system:

1. User logs on to approval system
2. User views list of pending purchase orders awaiting their approval
3. User selects a purchase order and views detail
4. User approves purchase order
5. System debits the departmental account by the amount on the purchase order
6. System forwards purchase order for fulfillment

Alternatives here would be for the user to reject the purchase order, for the debit to fail, or for the forwarding of the purchase order to fail. Steps 5 and 6 suggest a transactional relationship between them, hence requiring their mapping in the solution domain to use a transaction. In the case of `ApprovalProcess`, the `approve()` method would need to be marked as requiring a transaction, as would the `Account`'s `debit()` method. Use of transactional message queues would ensure that the move of the purchase order between the 'pending' and 'approved' queues formed an atomic operation with the debit.

The key to transactions is to keep them as short as possible. There is no need for all of the processing performed by `ApprovalWorkflow` or `ApprovalProcess` to be transactional. Indeed, this would have the effect of slowing down the system unnecessarily. The primary goal of transactions is to maintain data integrity. This requires that changes are isolated from each other and is essentially achieved by locking some of the underlying data. The more data is locked, the more likely it is that another transaction will have to wait until that lock is released. Although you can play many games with isolation levels, the bottom line is that more transaction use increases resource contention. Down this road lie timeouts and unhappy users. The only real solution to this is to keep transactions as short as possible. Therefore you should only use transactions where required, and ensure that transactional methods do not perform lots of unnecessary processing. Short transactions and rapid resource recycling are the two major factors in middle-tier scalability.

Lessons Learned

We could potentially follow the POSystem application through to the delivery of the order to the supplier system and so on. However, this would not really bring in any major new design issues. At this point, we can take stock of the main points to be drawn from the scenario and expand on these where appropriate.

Use Model-View-Controller (MVC) for User Interaction

MVC provides us with a very good micro-architecture for separating data from presentation and presentation from workflow. Although there are other architectures possible here, MVC is flexible and robust. Also, since it is widely understood, use of it will aid maintainability of your application.

JSP Design Principles

Ironically, as discussed in Chapter 14, the key to JSP design is to include as little of the 'J' element (Java) as possible. Any JSP that produces output should have its Java content minimized, to make it editable by the ordinary human beings who deal with page design and layout. Like normal HTML pages, JSPs should make use of directives to include other files (JSP or HTML) to create a common look to the site of which they form a part. This type of JSP, which forms an output template, is often called a **presentation component**. The only Java content in this type of page should be code for formatting data to be output as part of the page (for example catalogue listing). Even then, this should be minimized using one of the approved code 'outsourcing' methods.

Most Java code should be 'outsourced' to JavaBeans or servlets. User input should be handled by a front component that will process it and encapsulate the results in a JavaBean; this front component can be either a servlet or another JSP. This processing can include access to databases or EJBs, to generate any data that will form part of the output to the client. The data can be passed to the presentation component as part of the session or request. The front component then passes control to the presentation component. This mechanism is a specific use of a convenient filtering technique. Servlets and JSPs can be chained together with each one processing the user's request before it is forwarded to the next level. This type of design can be used to address issues such as those found in the 'Decorator' or 'Chain of Responsibility' patterns.

> *Note that JavaBeans can also be used as wrappers for EJBs. This allows you to encapsulate all the EJB access code within a JSP-friendly package.*

You can also use custom tags to remove code from JSPs, as described in Chapters 12 and 13. Although they require more work than using servlets and JavaBeans, they are more reusable and it is easier for page designers and their tools to handle the tags.

Choosing an Appropriate Data Format

Different formats can be used when passing data between different layers and components. In the POSystem, serialized JavaBeans were used to pass data between the EJB tier and the web tier, while XML was the chosen format when building messages to pass via JMS. Both of these choices could be reversed if other forces were at work on the system. If we knew that the application would have CORBA clients, or might need to interact with external systems, then XML would be a better choice for passing back data-snapshots from EJBs. Similarly, in an all-Java system there is no real reason (apart from future-proofing) not to use serialized JavaBeans as a standard data format.

One option not covered so far is the use of the `RowSet`s as seen in Chapter 3. `RowSet`s were introduced in JDBC 2.0 as a handy way of representing tabular data. A `RowSet` appears as a JavaBean, and can be used while disconnected from the database. It is also `Serializable`, so that it can be passed in an RMI method call. While it can be very useful in certain circumstances, passing a `RowSet` is not compatible with creating typesafe methods. Similarly, a field in a `RowSet` is potentially more open to incorrect interpretation than a property of a JavaBean or an element or attribute in XML. Passing data in `RowSet`s mirrors the popular use of `RecordSet` in ADO (ActiveX Data Objects) in Microsoft Windows DNA applications. In both cases, it tends to lock the architecture into one technology.

Reduce the Amount of Data Passed

One way to improve performance is to reduce the amount of data passed between components. We have seen that property-based programming is inadvisable. Even when using best practices such as returning a `Collection` of JavaBeans containing a snapshot of entity bean data, care should be taken to pass back only that data which is required by the client.

Using a session façade for entity beans also presents an opportunity to use the 'Batch Method' distributed pattern/idiom. Rather than the session bean returning a set of JavaBeans to be operated on by the client, the operation that the client wishes to perform can be migrated into the session bean. The iteration through the beans then takes place on the server, with only the results being passed back to the client. This reduces network traffic, albeit at the cost of some server processing overhead.

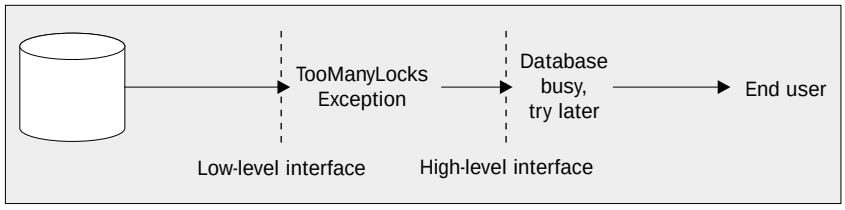
When using a session façade, it is always worth asking what you gain from the underlying entity bean. Would it be better to access the database directly if no other class in the system uses it?

Design Interfaces for Distribution

The design of interfaces between components is an art in and of itself, though there are common guidelines that help to shape good interfaces. The methods in an interface should form a cohesive set, rather than a disparate collection. The methods themselves should be meaningful operations, not just a set of property accessors. As in all good design, minimalism is a good principle, as long as it does not create too much extra work for the user of the interface. (A certain amount of de-normalization is allowed!) An example of this could be where there is a single method that takes many parameters. If you can identify a set of common tasks for which there are various default values, you could create a method for each of these tasks. From a purist point of view, these extra methods may seem wasteful, but from a practical point of view it can save much unnecessary client code for creating or specifying empty or standard parameters.

When creating distributed systems, such as J2EE applications, you must design the interfaces between components to take the distribution into account. Care must be taken when deciding whether to pass objects by reference or by value. Passing by reference is flexible, but can lead to increased overhead through remote calls. Passing by value makes for local interaction, but is not always the solution since it can cause problems for non-Java clients. If only a small part of the data is required, passing an object by value can lead to more overhead than a handful of distributed calls. In addition, passing by value is not appropriate for fast-changing data.

One issue that is seldom highlighted in interface design is the use of exceptions. Exceptions should be designed at the same time as the rest of the application, to reflect anticipated error conditions (`BudgetLimitExceeded`, `ProductNotFound`, etc.). These business-level exceptions should be thrown, rather than low-level ones. It is particularly important not to bundle everything as a `RemoteException`. Exceptions should be layered, so that different tiers throw exceptions that are relevant to them, and the tiers that catch them should either handle the exception or generate an exception that makes sense to *their* clients, as shown:



Use Messaging for Asynchronous Operations

In many applications, certain operations will take a long time to complete, and for the application to move on the operation must be made asynchronous. The messaging paradigm provides a simple way to design asynchronous operations. The use of messaging to defer slow operations is a distinct benefit for scalability. Obviously this must be used with care, since it is no good having a very scalable system that does the wrong thing!

Messaging is also a useful tool at times when the target server is not available, either by accident (server crash) or design (maintenance). In this way it brings a level of robustness to the application that is difficult to achieve when using exclusively synchronous mechanisms. Messaging does have downsides: it is more difficult to obtain a return value from the asynchronous processing, and transactions do not pass through the messaging system.

Messaging is not the only way of achieving asynchronous operations, since e mail and shared databases can also be used. However it does provide a common and flexible paradigm that is useful for integration with other systems.

Plan Transactions

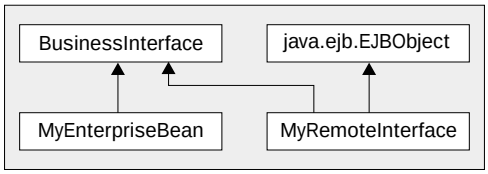
Systems should only use transactions as required; they can be discovered from use cases and designed in as appropriate. The time spent in transactions should be minimized as far as possible, to avoid resource contention. Failure to plan transactional requirements can lead to buggy or slow systems.

Beyond the Purchase Order System

Design always occurs within a context. The context we have used so far has been the purchase order system shown at the start of the chapter. However, there are many mechanisms and patterns that are useful in the design of J2EE systems that are not required or not used in the scenario. In this section, we will look at some of these and the contexts in which they may apply.

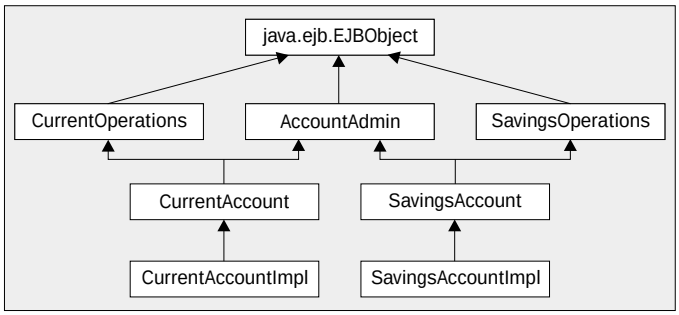
EJB Interface Design

When writing an Enterprise JavaBean, it is a bad idea for the bean to implement its remote (business) interface. This would allow a reference to the bean itself to be accidentally passed back to a client, rather than passing the associated `EJBObject`. However implementing the remote interface is tempting, since it allows you to pick up any incorrect method signatures on the bean before the container's introspector does. To achieve this end, you can define a non-remote version of your business interface that can be implemented by the bean, and then create your bean's remote interface by inheriting from both the non-remote business interface and from `EJBObject`. The diagram below shows the inheritance hierarchy of the interfaces. Note that the methods on your non-remote business interface will still have to be declared to throw `RemoteException` in order to work correctly in the remote version:



Unlike Microsoft's COM, or the forthcoming CORBA Components environment, Enterprise JavaBeans have only a single business interface. This means that there are issues with the design of EJBs that must be worked around. If the design of the bean suggests that it should implement two separate interfaces then this requirement can be addressed by using inheritance.

The next figure shows a potential inheritance hierarchy for two account interfaces, `CurrentAccount` and `SavingsAccount`. Both will have some common administrative operations, but they may well differ in the rest of their operations. Rather than just grafting a list of administrative methods into each interface and risking them getting out of sync, each interface can inherit from two others – the `AccountAdmin` interface containing the common methods, and an interface containing the account-specific methods, such as `CurrentOperations`. This would allow a client to narrow a `CurrentAccount` reference to either type `AccountAdmin` or `CurrentOperations`. The rest of the client code could then be written as if these were separate interfaces, and so type safety could be assured:



Note that this can only be done for the remote interfaces of EJBs. Home interfaces will need to return different types from their methods, and hence cannot be used in this way.

A final consideration for EJB interfaces is the consequence of using role-based discovery of interfaces. The identification of a role during analysis will often lead to the discovery of an interface containing methods specific to that role. An example of such an interface is the `AccountAdmin` interface in the figure above, which should be restricted to an administrator role. There are two possible routes here:

- You could implement the interface on an entirely separate bean. However, this would not be desirable if the methods on `AccountAdmin` and `CurrentOperations` acted on the same underlying data.
- Alternatively, you could implement the type of inheritance hierarchy shown above, and use the security attributes in the deployment descriptor to apply the correct security to this virtual interface. Although this is slightly unwieldy, it is the only safe alternative in the absence of multiple interfaces.

Distributed Events

As discussed previously, efficient operation is achieved by splitting most state between the Web and EJB tiers. A data bean can act as a read-only representative of the data in the associated EJB. One consideration here is what to do when the data being represented changes. The MVC pattern relies on the model to inform the view when its data changes. When the data is manipulated locally (in the same JVM), the delegation event model works fine for this: the model can fire some form of change event to the view, which would implement the appropriate listener interface for that event. The Swing API uses this form of event mechanism as part of its MVC implementation.

The issue here is that the data model is not local to the view: the view is in the web (or client) tier, and the model is in the EJB tier. There is no pre-supplied Java distributed event mechanism. However, there is enough of a framework to create your own. RMI allows you to set up callbacks from server to client, and these can be used as the basis for a distributed version of the delegation event model, with the server implementing a registration interface and the client (or view) adding itself as a listener. When its data changes, the server can inform all of its listeners of the change, and the views can update themselves based on this new data. If you wish to implement this type of framework, the Jini Distributed Event Specification, available from Sun's web site, provides a good basis.

One thing to note here for web-based interfaces is that they cannot change the browser's screen in an unsolicited fashion. Only when the user next submits a form or clicks on a link will the server have a chance to update the screen contents. (I am intentionally ignoring 'timed pull' here.) The Java Pet Store that comes with the J2EE Blueprints provides an interesting slant here. It allows web tier components to register their interest in particular data models with an instance of a local class called the `ModelManager`. The models themselves live on the EJB tier. Rather than setting up a distributed event system, each logical action submitted to the workflow on the EJB tier will return a list of the models that it has updated. The `ModelManager` instance will then inform the appropriate listeners in the web tier of these changes, and this ensures that the data beans in the web tier are up to date the next time the view is refreshed.

Another alternative for a distributed event system is to use asynchronous messages. Currently (pre-EJB 2.0) this has some issues when used with EJBs. If your EJB is the target of the event, it must wait for event messages to appear on the queue by implementing a listener interface. However this is not a good idea, since the EJB may have timed out by the time the message arrives. Therefore the event must actually be delivered to a helper object that can be stored via JNDI and retrieved later if required, and this helper object must be able to invoke the EJB and deliver the message to an appropriate method. This is rather inelegant and does not scale too well without a lot of work. EJB 2.0 promises message-driven EJBs, which will address this problem.

JMS itself can be used to implement various message-based or event-based patterns, though the functionality of interest will depend very much on the forces at work on the particular sub-system you are trying to design. A message queue can be used to pass messages from a producer to one or more consumers. Using point-to-point messages ensures a 1-to-1 relationship between the producer and consumer (although the behavior in the case where multiple consumers are silly enough to register for messages from the same queue is somewhat vague). Point-to-point would be good where a producer is creating items of work that must be processed by a consumer. Alternatively, when dealing with the distribution of information (such as the ubiquitous stock quotes), a publish-and-subscribe model can be used. In this case, multiple clients will receive the same information by registering for messages on a specific topic.

Messages can be made persistent if the application will have problems should they be lost in transit. Similarly, consumers can request that a server retain a copy of any publish-and-subscribe messages that have arrived for them when they were disconnected. These will be delivered when the consumer reappears. All of this helps the robustness of an application.

Working with Databases

There are many clever things you can do with databases that are beyond the scope of this chapter. If you are seriously into Java and database interaction you should read Chapter 3 or refer to something like *Database Magic*, with Ken North, Prentice Hall, ISBN 0-13-647199-4. However, from an overall design standpoint there are one or two principles to consider.

Firstly, don't be a Java snob. EJBs are great and they do a fine job of providing scalable data and business workflow. However, if performance is a high priority, why not let the database do as much work as possible? It is more efficient to run code inside the database than it is to extract the data, work on it and then put it back. Although embedding a stored procedure in your database and calling it to perform the work may not be as flexible, portable, or as cool using an entity EJB, you may find that it works many times faster and so could be a better fit for the forces on your application.

If you are using entity EJBs, try to avoid fighting the database. If you find you are querying many tables to build your entity's state then you may have a serious impedance mismatch between your bean and database. You might want to consider denormalizing your database somewhat to ease the fit with your entity, or creating one or more views that are a better match.

In general, it is worth spending a lot of time on your data design, as this tends to move a lot more slowly than the middle tier or client tier. If you get it wrong it can require serious rework of the other tiers.

State Management

State management is sometimes confused with transactions, especially if people have read some of the documentation on Microsoft Transaction Server (MTS). In MTS there is a tight coupling between transactions and stateless operation. This led many people to imply that transactions were the key to scalability. While transactions certainly help, the key aspect for scalability is state management.

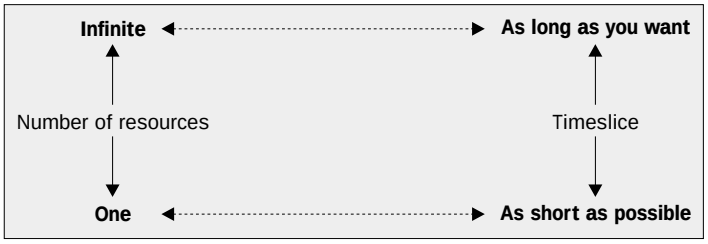
State management has two parts to it:

- Synchronization (in the sense of consistency and persistence) of data
- Recycling of resources

EJBs make data synchronization reasonably simple; if you correctly handle the lifecycle methods of your chosen EJB type you cannot go far wrong. In the web and client tier, data tends to be more disposable, so you must take more responsibility for its persistence if this is required.

Much is spoken and written about the choice between stateless and stateful middle tier components. Each type of component has its own uses, and it would be foolish to use one particular type simply to conform to dogma. At the end of the day, you could model a typical `Account` bean either as an entity or as a stateless session bean that retrieves its data from a database at the start of each method. Only the interface will change to protect the innocent!

The final word is on recycling of resources; this is an important concept in all tiers. If you have an infinite number of resources available, then it does not matter how long a component holds on to a particular resource since other components can retrieve another one out of the infinite pool. As the number of resources drops, the amount of time a component holds the resource becomes more critical. This relationship is shown below:



Most of our applications have more than one resource, but it is still important to recycle resources as quickly as possible to aid scalability. The key is to use resources from resource pools, since these are allocated quickly. That means that you do not have to 'take the hit' of resource acquisition at initialization time, but can acquire and release resources when you need them throughout your component code. You should acquire a resource just before you need it, and release it as soon as possible afterwards. It may even be possible that another component you call could reuse a resource you have released earlier in the method.

In the EJB tier, there is a tendency to be less concerned about resource recycling, since it is presumed that the container will handle all of this for you. However, the principle of releasing resources as quickly as possible still applies. See the next chapter for more discussion on performance and scalability.

On Architecture and Process

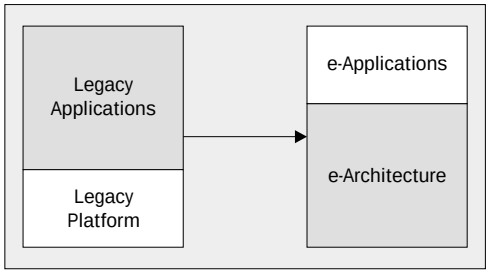
Throughout this chapter I have stressed the need to judge architecture and design in context. The requirements that different types of application make on their application architecture will vary, for example:

- **e-Commerce apps:** Many clients, lots of reading, less updating, low contention, large working set
- **Banks and ATMs:** Many clients, low concurrency, high isolation level, no working set

The requirements of your application will depend very much on the functionality you are delivering, and this in turn will have a large influence on the decisions you make about your architecture. The most common application types give rise to architectural templates. The architecture used by the POSystem (3 tiers, thin client, etc.) is one of the most common, since it fits the web-based e-commerce style applications currently in vogue. Other architectural templates exist for differing types of application.

One of the most common non-functional requirements for systems is 'short time to market'. Systems are required 'yesterday', but with higher functionality, scalability, availability, etc. than ever before.
One concession to this is the increased use of common off-the-shelf (COTS) components where available. Build versus buy has always been a tradeoff to be made, but these days few organizations have the ability to build everything from scratch in the required timeframe. What is required is a solid architectural base on which to build applications.

In the past the application was king, to the extent that it relied only on the basic functionality of its underlying operating system or environment. This could be seen from the efforts required to port applications between platforms. Environments such as J2EE help to raise the bar in this respect. Much of the functionality that was previously part of the application can now be delegated elsewhere – to horizontal layers or vertical services. This gives rise to the 'tear off application', since the application code itself forms only a small part of the overall functionality. Since there is a lower investment in the application it can be re-written more often, and thrown away more easily when it becomes outdated. e-Commerce applications are typical of this sort of application (or should be). This transition is shown in the figure below:



Another area where 'Internet time' has imposed time constraints is the development process. Before I go further, let me state here and now that I am not a great process person. I tend to focus on the technologies and architecture, rather than the finer points of the journey from requirements capture to implementation. However, it is interesting to observe how the approach to the development cycle has changed in the face of the Web and how this relates to design.

In one report on e-commerce, Gartner states that the transition to e-commerce is, "a programme, not a project". This reflects the realities in a world of rapidly changing business requirements. In such an environment, a commercial application cannot remain largely the same for two or three years after delivery. The changes required of the application in that timeframe will be significant – not simply bug fixes and usability improvements. In order to avoid becoming a legacy application, it must constantly evolve to meet changing demands.

All developers are familiar with the multiple iterations of code-compile-test that take place during implementation. To evolve the application, all parts of the analysis and development process become iterative. This has been taken on board by all forms of process, from high ceremony to low, from the Rational Unified Process (RUP) through the Microsoft Solutions Framework (MSF) and on to eXtreme Programming (XP).

So what does this mean for design? It means that you should approach design as an ongoing process, not as a once-only event. Approaching it as such can actually reduce the cost and impacts of incorrect design decisions. Over the life of any application, the technological landscape will change. This landscape is part of the context in which design decisions are made, and decisions made on the basis of a previous technology landscape may look stupid in a new one. What is needed is an acceptance that change will occur, and that it must be factored into the application lifecycle.

The acceptance of change leads to projects where functionality is handed off in stages, where the design evolves as the project progresses, and where refactoring is a part of everyday life (see *Refactoring: Improving the Design of Existing Code*, Fowler *et. al.*, by Addison-Wesley Publishing Co. ISBN 0-201-48567-2). Changes in technology and implementation issues should feed back up the chain, so that their implications can be taken into account in the mapping of the problem model onto the solution model. (Frank Buschmann uses a yo-yo as an analogy for this effect.) Although an open and flexible architecture is a good asset, take care not to spend too much time turning simple classes into libraries or frameworks that may never be exploited. When considering such exercises it is useful to bear in mind the eXtreme Programming principle of YAGNI, or "You Ain't Gonna Need It".

Summary

For many years I have been involved in the evaluation of new technologies. At the beginning, I tended to focus on the technology itself and the various bells and whistles it provided. Over time, it dawned on me that 90% of the developers in the world would never use these bells and whistles.

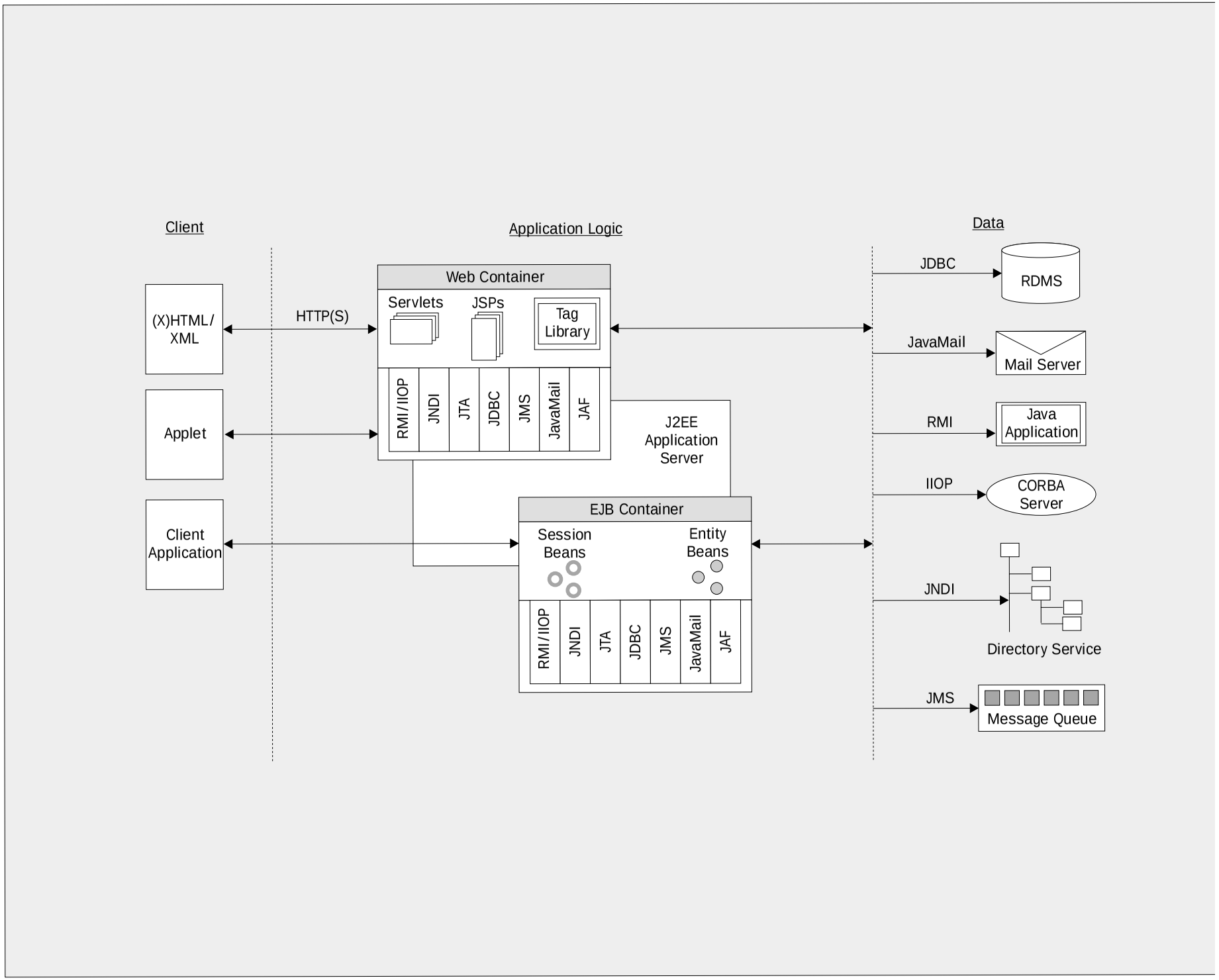
> **What they wanted to do was to use the technology to solve their business problems.**

To do that, they needed to understand the technology and apply it in their own particular context. The key really is to ensure that programmers use the core features of the particular technology appropriate to their context. The technology itself is of little use unless it is correctly applied. I hope that you have found some familiar context in this chapter, and will be able to apply some of the principles described.

In this chapter we have examined aspects of J2EE design, specifically:

- The relationship between design and architecture
- How the context provided by a particular type of application can affect the design decisions made
- Some of the issues typically found when designing a common J2EE-based application
- Some strategies for dealing with issues such as distribution, state management, and transactions in J2EE application design
- How different J2EE technologies may be applied to solve different design problems

The next chapter looks at the vital topics of performance and scalability in J2EE applications.


Client
Application Logic
Data
(X)HTML/
XML
HTTP(S)
Applet
Client
Application
Web Container
Servlets
JSPs
Tag
Library
RMI/IIOP
JNDI
JTA
JDBC
JMS
JavaMail
JAF
J2EE
Application
Server
EJB Container
Session
Beans
Entity
Beans
RMI/IIOP
JNDI
JTA
JDBC
JMS
JavaMail
JAF
JDBC
RDMS
JavaMail
Mail Server
RMI
Java
Application
IIOP
CORBA
Server
JNDI
Directory Service
JMS
Message Queue